



Assignment – Tech

[Kiddo](#) is a Q-commerce platform for kids & baby essentials, focused on making parenting easier through convenience, speed, and trust.

CONTEXT

At our company, the mobile app homepage is highly dynamic and volatile. We run frequent live marketing campaigns (e.g., Diwali, New Year, Summer Sales) with a hard constraint of zero App Store / Play Store release cycles.

To achieve this flexibility, we leverage a robust Server-Driven UI (SDUI) architecture. The React Native frontend acts as an entirely "dumb" rendering engine that parses a complex, deeply structured JSON payload, dynamically constructs the screen tree, injects operational themes, and maps declarative actions safely to local client execution blocks.

CHALLENGE

Your core task is to design and build a highly performant, production-ready, configuration-driven React Native homepage renderer.

The codebase must elegantly parse a massive mock JSON payload and flawlessly render heterogeneous UI component blocks, with deep engineering emphasis placed squarely on Dynamic Collections, structural resilience, and Live Campaigns management.

STACK TO KNOW

To ensure maximum baseline performance and architectural alignment, your submission must leverage:

- Framework: React Native (Expo or Bare Workflow are both acceptable).
- Language: TypeScript configured in Strict Mode (highly preferred).
- List Rendering: @shopify/flash-list (Highly Recommended for high frame-rate rendering) or optimized native FlatList.
- Animations: lottie-react-native or highly optimized native WebP/GIF parsing modules.
- State Management: Local collocation state paired with React Context API, Zustand, or custom native refs architected cleanly to fully bypass global tree re-renders.



1. CORE REQUIREMENTS:

A. The JSON Schema & Component Registry

Construct a local mock JSON file simulating the heavy operational payload delivered by our production backend gateways. Your React Native client engine must ingest this payload dynamically and use a declarative Component Registry (implementing a scalable Factory Pattern) to map backend nodes to active layout definitions.

The inbound collection payload must securely represent structural UI blocks including:

- **BANNER_HERO:** A fluid, full-width promotional graphic card built to present immediate heroic marketing focus.
- **PRODUCT_GRID_2X2:** A standard, balanced 2x2 grid structure parsing and nesting simple product catalog item components.
- **DYNAMIC_COLLECTION:** A horizontal scrolling carousel layer grouping elements tied tightly to a server-pushed contextual marketing theme (e.g., 'Summer Essentials', 'Snacks under ₹99').

⚠ RESILIENCE CRITICAL RULE: Your engine must defensively match incoming string signatures to registration components. If an unsupported structural type is parsed (e.g., 'NEW_COMPONENT_V2'), your renderer must fail gracefully, drop the unrecognized node quietly, and preserve the total stability of the surrounding view tree.

B. Dynamic Collections & Virtualization Boundaries

You must successfully compose the DYNAMIC_COLLECTION layout abstraction. This component instantiates a horizontal FlatList completely nested deep inside the core master vertical tracking feed layer.

Constraint: You must actively architect the interaction maps to ensure that dragging or scrolling horizontally across this collection does not break, stutter, or drop the vertical velocity momentum of the primary interface list, nor scale up heap allocations or memory leaks over heavy scrolling.



C. The Universal Action Dispatcher

Every component block configuration transmitted from the server will have a standardized action schema object bound to its interactive layout nodes. For instance:

```
"action": { "type": "ADD_TO_CART", "payload": { "id": "123" } }  
"action": { "type": "DEEP_LINK", "payload": { "url": "/category/snacks" } }
```

You must implement a structured, centralized `handleAction(actionObj)` dispatcher module. The atomic layout components must remain decoupled and fully ignorant of business validation logic, executing only raw interface triggers to pass context to this coordinator.

D. High Frame-Rate Optimization (Strict Rule)

You are required to process and stream the entire operational JSON layout profile cleanly inside a single, singular vertical `FlatList` or `FlashList` framework. Component processing pipelines are heavily analyzed based on sustained frame production rate, precise index stability mapping via strict `keyExtractor` algorithms, and the structural application of `React.memo` boundaries.

2. Advanced Deliverables: Live Campaigns & Local State Optimization

To demonstrate engineering capacity matching enterprise hyper-growth platforms like Blinkit or Zomato, implement the following complex live campaign architecture mechanisms:

A. Remote Overlay Contexts (The Campaign Engine)

The upstream service boundary emits contextual live overlay configurations corresponding to operational festival metrics. You must generate and bundle functional local payload profiles representing three highly distinct campaigns, showing the client's capabilities to dynamically switch runtime visuals instantly without requiring app binary updates:



Campaign Context	Theme Injections	Components & Reference Assets
1. 'Back to School' Mega-Sale	Injects an intense, high-contrast bright yellow and primary blue context skin configuration theme object.	• Parses a remote Lottie asset file mapping paper airplanes / falling pencils. • Displays a dedicated horizontal target row for 'Lunchboxes & Bags'.  Reference Attachment: [Attach/Embed Video 1 Here]
2. 'Summer Playhouse' Festival	Injects a cool, fluid ocean blue color tracking palette environment directly into layout layers.	• Spawns an animated interactive 'Water Splash' / 'Beach Ball' WebP overlay. • Integrates specialty event booking components for 'Petting Zoo Tickets'.  Reference Attachment: [Attach/Embed Video 2 Here]
3. 'Mystery Gift Carnival'	Injects an explicit carnival red festival theme across baseline layout surfaces.	• Renders complex, overlapping burst popping confetti layer canvas elements. • Interactive product row targets bind deep macro actions like APPLY_MYSTERY_GIFT_COUPON.  Reference Attachment: [Attach/Embed Video 3 Here]

For whichever campaign context is active, the engine must safely resolve and scale this layout overlay node:

```
{
  "type": "FULL_SCREEN_OVERLAY",
  "animation_url": "https://assets.example.com/confetti_carnival.json"
}
```

Overlay Engineering Constraint: You must render this graphic wrapper animation layout



cleanly overlapping the full-screen interactive space. Crucially, apply `pointerEvents="none"` context handles so users continue to click, slide, and fully interact with the underlying operational application layout elements without input occlusion. Remote campaign media files must be parsed via an efficient cache pipeline.

B. Over-The-Air (OTA) Runtime Theming Injection

The core layout payload provides a functional structural theme matrix:

```
"theme": { "primary": "#FF9933", "background": "#FFF5E6" }
```

You are required to map this dynamic structural palette block explicitly into a React Context Provider wrapping the root engine node. All nested operational child component skins (e.g., call-to-action buttons, border limits, headers) must sample this context to shift interface visual configurations dynamically.

C. Local State Collocation (The Rendering Cycle Challenge)

When an interactive user fires an `ADD_TO_CART` payload handle on an element within a `DYNAMIC_COLLECTION` list layer or structural `PRODUCT_GRID` card, the global interface cart increment counter state tracking must reflect updates instantly.

🧠 ARCHITECTURAL RENDERING MANDATE: Mutating the cart selection quantity on one single atomic product card **MUST NOT** run execution updates across, nor trigger re-renders down, the other 30+ layout engine blocks mounted inside the vertical feed framework. Submissions must demonstrate thorough command of component memo isolation boundaries and optimized rendering ticks.

Candidate Evaluation Criteria

- **Architectural Cleanliness:** Evaluation of the structural design patterns governing the Component Factory Registry. Codebases relying on brittle, highly unscalable switch blocks will place lower than submissions relying on dynamic, clear hash-maps or runtime registry configuration classes.
- **Sustained Frame Performance:** Assessment of optimization structures, layout execution times, the absence of virtualization drop ticks, and the professional application of React.memo boundaries and specialized container list performance handles.
- **TypeScript Strategy:** Evaluation of schema interface type strictness, full structural type definitions for unpredictable dynamic layouts, action contracts, and complete system type-safety boundaries.
- **System Defensive Resilience:** Verification of processing safety when handling corrupt metadata, unexpected payload attributes, or unmapped server component layouts, ensuring the client framework isolates faults beautifully.